

リバーシ mian.dart について

1. プログラム概要

main.dart は、本システムのユーザーインターフェースを管理するプログラムである。Flutter を用いてゲーム画面を構築し、リバーシ盤面の表示やプレイヤーの操作受付を行う。

また、現在の手番やスコア、ゲーム結果などの情報を表示するとともに、ゲームモードの切り替えやリセット機能などを担当している。

さらに、AI 対戦時には AI の思考開始タイミングを制御し、思考中の操作制限を行うことで、円滑なゲーム進行を実現している。

logic.dart で実装されたゲームロジックと連携し、ゲームの進行状況に応じて画面表示を更新する役割も担う。

2. プログラム内容(抜粋)

2.1. AI 着手制御機能

プレイヤーの着手後に現在の手番を判定し、AI の手番である場合は自動的に AI の着手処理を実行する機能である。

AI の難易度に応じて異なる着手アルゴリズムを呼び出し、選択された手をゲームロジックへ反映する。また、AI 思考中はユーザー操作を無効化することで誤操作を防止している。

[解説]

・ 3 行目:現在の手番が AI であるかを判定し、人間の手番では処理を実行しない。

```
1 void _triggerAiMoveIfNeeded() async {
2   if (_game.gameMode != 0 && !_game.isGameOver()) {
3     if (_game.currentPlayer != _game.aiPlayer) return;
4     setState(() {
5       _isAiThinking = true;
6     });
7
8     Future.delayed(const Duration(milliseconds: 1000), () {
9       if (!mounted) return;
10      List<int>? aiMove;
11
12      if (_game.gameMode == 1) {
13        aiMove = _game.getWeakAiMove();
14      } else if (_game.gameMode == 2) {
15        aiMove = _game.getStrongAiMove();
16      } else if (_game.gameMode == 3) {
17        aiMove = _game.getExpertAiMove();
18      }
19
20      if (aiMove != null) {
21        _game.playTurn(
22          aiMove[0],
23          aiMove[1],
24        );
25      }
26    });
27  }
```

```

26     );
27   }
28
29   setState(() {
30     _isAiThinking = false;
31   });
32   _triggerAiMoveIfNeeded();
33 });
34 }
35 }

```

```

1  @override
2  Widget build(BuildContext context) {
3    // 現在の手番に応じて背景色を変更
4    final Color backgroundColor = _game.currentPlayer == 1
5      ? Colors
6        .green
7        .shade50 // 黒の手番
8      : Colors.blue.shade50; // 白の手番
9
10   return Scaffold(
11     // アプリ上部のヘッダー
12     appBar: AppBar(
13       title: const Text('リバーシ'), // タイトル表示
14       centerTitle: true, // タイトルを中央揃え
15       backgroundColor: Colors.transparent, // 背景透明
16     ),
17
18     // 背景色をアニメーション付きで変更
19     body: AnimatedContainer(
20       duration: const Duration(milliseconds: 500), // アニメーション時間
21       curve: Curves.easeInOut, // 緩やかな変化
22       color: backgroundColor, // 手番に応じた背景色
23       // スマートフォンのノッチ等と重ならない領域
24       child: SafeArea(
25         child: Column(
26           children: [
27             const SizedBox(height: 12), // 上部余白
28
29             _buildScoreBoard(), // スコア・操作パネル表示
30
31             const SizedBox(height: 12), // 間隔
32
33             Expanded(child: Center(child: _buildBoard())),
34           ],
35         ),
36       ),
37     );
38 };
39 }
40
41 // スコアボードおよびゲーム状態表示エリアを生成
42 Widget _buildScoreBoard() {
43   // 現在の手番に応じて石の色を決定
44   final Color turnColor = _game.currentPlayer == 1
45     ? Colors.black
46     : Colors.white;

```

```

47
48 // 現在の手番プレイヤー名を表示するテキスト
49 String turnText = _game.currentPlayer == 1
50   ? "${_game.blackPlayerName} (黒) の番"
51   : "${_game.whitePlayerName} (白) の番";
52
53 // AI思考中は専用メッセージを表示
54 if (_isAiThinking) {
55   turnText = "AI思考中...";
56 }
57
58 // ゲーム終了判定
59 final bool isOver = _game.isGameOver();
60
61 // 黒石の数を取得
62 int black = _game.countStones(1);
63
64 // 白石の数を取得
65 int white = _game.countStones(2);
66
67 // 石数を比較して勝敗メッセージを決定
68 String resultText = black > white
69   ? "🏆 黒の勝利!"
70   : white > black
71   ? "🏆 白の勝利!"
72   : "🤝 引き分け!";
73
74 // 勝者に応じて結果表示エリアの背景色を変更
75 Color winnerContainerColor = black > white
76   ? Colors.black
77   : white > black
78   ? Colors.white
79   : Colors.grey.shade300;
80
81 // 勝敗表示の文字色を設定
82 Color winnerTextColor = (black > white) ? Colors.white :
Colors.black;
83
84 return Column(
85   children: [
86     // 手番表示と勝敗表示を行うエリア
87     AnimatedContainer(
88       duration: const Duration(milliseconds: 300), // 表示切替時のアニメーション時間
89       // ゲーム中と終了時で余白サイズを変更
90       padding: EdgeInsets.symmetric(
91         horizontal: isOver ? 32 : 24,
92         vertical: isOver ? 14 : 8,
93       ),
94
95       decoration: BoxDecoration(
96         // ゲーム終了時は勝者色, 通常時は半透明の白背景
97         color: isOver
98           ? winnerContainerColor
99           : Colors.white.withValues(alpha: 0.7),
100         borderRadius: BorderRadius.circular(30), // 角を丸くする
101         boxShadow: // ゲーム終了時は影を強調
102           isOver

```

```

103         ? [
104             BoxShadow(
105                 color: Colors.black26,
106                 blurRadius: 10, // 影のぼかし量
107                 offset: const Offset(0, 4), // 影の位置
108             ),
109         ]
110       : const [
111         BoxShadow(
112             color: Colors.black12,
113             blurRadius: 4,
114             offset: Offset(0, 2),
115         ),
116       ],
117     ),
118     // 横方向に要素を配置
119     child: Row(
120       mainAxisAlignment: MainAxisAlignment.min,
121       children: [
122         // ゲーム進行中のみ現在の手番の石を表示
123         if (!isOver) ...[
124           AnimatedContainer(
125             duration: const Duration(milliseconds: 200), // 石の切替
126             アニメーション
127             width: 24, // 石の幅
128             height: 24, // 石の高さ
129             decoration: BoxDecoration(
130               shape: BoxShape.circle, // 円形
131               color: turnColor, // 黒番なら黒, 白番なら白
132               border: Border.all(
133                 // 白石の場合のみ外枠を表示
134                 color: _game.currentPlayer == 1
135                   ? Colors.transparent
136                   : Colors.black26,
137                 width: 2,
138               ),
139             ),
140           ],
141         const SizedBox(width: 12), // 石と文字の間隔
142       ],
143     ),
144     // 手番または勝敗結果を表示
145     Text(
146       // ゲーム中は現在の手番, 終了時は勝敗結果を表示
147       isOver ? resultText : turnText,
148       style: TextStyle(
149         // 終了時は文字サイズを大きくして強調
150         fontSize: isOver ? 22 : 18,
151         fontWeight: FontWeight.bold, // 太字
152         // 勝敗表示時は背景色に応じて文字色を変更

```

```

161         color: isOver ? winnerTextColor : Colors.black,
162     ),
163 ),
164 ],
165 ),
166 ),
167
168 // パス発生時のメッセージを表示
169 // ゲーム終了後は表示しない
170 if (_game.passMessage.isNotEmpty && !isOver)
171   Padding(
172     // 上方向に少し余白を追加
173     padding: const EdgeInsets.only(top: 4),
174
175     child: Text(
176       // 「黒は置けないためパス」などを表示
177       _game.passMessage,
178
179       style: const TextStyle(
180         color: Colors.red, // 注意表示のため赤色
181         fontSize: 14,
182         fontWeight: FontWeight.bold,
183       ),
184     ),
185   ),
186
187   const SizedBox(height: 8), // 間隔
188   // 黒石と白石の数を表示するエリア
189   Row(
190     mainAxisAlignment: MainAxisAlignment.center,
191
192     children: [
193       // 黒石数表示
194       _buildSmallScoreBadge(player: 1, count: black),
195
196       const SizedBox(width: 20),
197
198       // 白石数表示
199       _buildSmallScoreBadge(player: 2, count: white),
200     ],
201   ),
202
203   const SizedBox(height: 12), // 間隔
204   // 操作パネル
205   // 戻るボタン・リセットボタン・ゲームモード切替を配置
206   Padding(
207     padding: const EdgeInsets.symmetric(horizontal: 16.0),
208
209     child: Wrap(
210       // ボタン同士の間隔
211       spacing: 8,
212       runSpacing: 8,
213
214       alignment: WrapAlignment.center,
215       crossAxisAlignment: WrapCrossAlignment.center,
216
217       children: [
218         // 解説ページへ戻るボタン
219         IconButton(

```

```

220         onPressed: () async {
221             // 解説ページのURLを生成
222             final Uri url = Uri.parse(
223                 'https://www.oit.ac.jp/labs/rd/rssrv/kobayashi-lab/~
takamiya/flutter/reversi.html',
224             );
225
226             // URLが開ける場合のみブラウザ起動
227             if (await canLaunchUrl(url)) {
228                 await launchUrl(url, mode:
LaunchMode.externalApplication);
229             }
230         },
231
232         icon: const Icon(Icons.logout), // 戻るアイコン
233         tooltip: '解説ページへ戻る',
234     ),
235
236     // ゲームリセットボタン
237     ElevatedButton.icon(
238         onPressed: () {
239             setState(() {
240                 // ゲーム状態を初期化
241                 _game.reset();
242
243                 // AI思考状態も解除
244                 _isAiThinking = false;
245             });
246
247             // AI戦の場合は初手AIを実行
248             _triggerAiMoveIfNeeded();
249         },
250
251         icon: const Icon(Icons.refresh), // リセットアイコン
252         label: const Text("リセット"),
253     ),
254
255     // ゲームモード切替ボタン群
256     ToggleButtons(
257         // 現在選択中のモードを表示
258         isSelected: [
259             _game.gameMode == 0, // 二人対戦
260             _game.gameMode == 1, // AI弱
261             _game.gameMode == 2, // AI強
262             _game.gameMode == 3, // AI最強
263         ],
264
265         // ボタン押下時の処理
266         onPressed: (int index) {
267             setState(() {
268                 // 選択されたモードに変更
269                 _game.gameMode = index;
270
271                 // 新しいモードでゲーム開始
272                 _game.reset();
273
274                 _isAiThinking = false;
275             });
276

```

```

277         // AI戦の場合はAI着手判定
278         _triggerAiMoveIfNeeded();
279     },
280
281     borderRadius: BorderRadius.circular(20), // ボタン角丸
282
283     constraints: const BoxConstraints(minHeight: 36,
minWidth: 68),
284
285     selectedColor: Colors.white, // 選択時文字色
286
287     fillColor: Colors.green.shade700, // 選択時背景色
288
289     children: const [
290         // 人対人モード
291         Text(
292             "二人対戦",
293             style: TextStyle(fontSize: 12, fontWeight:
FontWeight.bold),
294         ),
295
296         // ランダムAI
297         Text(
298             "AI (弱) ",
299             style: TextStyle(fontSize: 12, fontWeight:
FontWeight.bold),
300         ),
301
302         // 重み付けAI
303         Text(
304             "AI (強) ",
305             style: TextStyle(fontSize: 12, fontWeight:
FontWeight.bold),
306         ),
307
308         // 探索AI
309         Text(
310             "AI (最強) ",
311             style: TextStyle(fontSize: 11, fontWeight:
FontWeight.bold),
312         ),
313     ],
314 ),
315 ],
316 ),
317 ),
318 ],
319 );
320 }
321
322 // 石の色と個数を表示するスコアバッジを生成
323 Widget _buildSmallScoreBadge({required int player, required int count})
324 {
325     return Row(
326         children: [
327             // 石を表す円
328             Container(
329                 width: 16,

```

```

329         height: 16,
330
331         decoration: BoxDecoration(
332             shape: BoxShape.circle,
333
334             // プレイヤーに応じて石の色を変更
335             color: player == 1 ? Colors.black : Colors.white,
336
337             border: Border.all(color: Colors.black38, width: 1),
338         ),
339     ),
340
341     const SizedBox(width: 6), // 石と個数表示の間隔
342     // 石の個数を表示
343     Text(
344         "$count個",
345         style: const TextStyle(fontSize: 14, fontWeight:
Widget FontWeight.bold),
346     ),
347     ],
348 );
349 }
350
351 // リバーシ盤面を生成
352 Widget _buildBoard() {
353     return Padding(
354         // 盤面の外側余白
355         padding: const EdgeInsets.all(16.0),
356
357         child: AspectRatio(
358             // 常に正方形の盤面を維持
359             aspectRatio: 1,
360
361             child: Container(
362                 // 盤面全体の外枠
363                 decoration: BoxDecoration(
364                     border: Border.all(color: Colors.black, width: 2),
365                 ),
366
367                 // 8×8のマスを生成
368                 child: GridView.builder(
369                     // スクロールを無効化
370                     physics: const NeverScrollableScrollPhysics(),
371
372                     gridDelegate: const
SliverGridDelegateWithFixedCrossAxisCount(
373                         // 8列構成
374                         crossAxisCount: 8,
375                     ),
376
377                     itemCount: 64, // 8×8=64マス
378
379                     itemBuilder: (context, index) {
380                         // 1次元インデックスを行番号へ変換
381                         int row = index ~/ 8;
382
383                         // 1次元インデックスを列番号へ変換
384                         int col = index % 8;
385

```



```

386 // 現在マスの状態取得
387 // 0:空き 1:黒 2:白
388 int stoneState = _game.board[row][col];
389
390 return GestureDetector(
391   // マスがタップされた時の処理
392   onTap: () {
393     // AI思考中または石が存在する場合は無効
394     if (_isAiThinking || stoneState != 0) {
395       return;
396     }
397
398     // 石を置けた場合
399     if (_game.playTurn(row, col)) {
400       // 画面更新
401       setState(() {});
402
403       // AI対戦時はAI着手判定
404       _triggerAiMoveIfNeeded();
405     }
406   },
407
408   child: Container(
409     // マスの見た目設定
410     decoration: BoxDecoration(
411       // リバーシ盤面の緑色
412       color: const Color(0xff0f7d32),
413
414       // マスの境界線
415       border: Border.all(color: Colors.black54, width:
416         0.5),
417     ),
418
419     child: Stack(
420       alignment: Alignment.center,
421
422       children: [
423         // 石の描画
424         ReversiStone(state: stoneState),
425
426         // 合法手の場所に黄色マーカーを表示
427         if (stoneState == 0 &&
428           _game.canPlaceStone(row, col,
429             _game.currentPlayer))
430           const CircleAvatar(
431             radius: 5,
432             backgroundColor: Colors.yellow,
433           ),
434       ],
435     ),
436   ),
437 ),
438 ),
439 ),
440 );
441 }

```

```

442 }
443
444 // リバーシの石を描画するウィジェット
445 class ReversiStone extends StatelessWidget {
446   // 石の状態 0:空きマス 1:黒石 2:白石
447   final int state;
448   const ReversiStone({super.key, required this.state});
449
450   @override
451   Widget build(BuildContext context) {
452     if (state == 0) return const SizedBox(); // 空きマスの場合は何も表示しない
453
454     final double turns = state == 1 ? 0.0 : 0.5; // 黒石なら0回転, 白石なら
180度回転
455
456     return FractionallySizedBox(
457       // マスの85%のサイズで石を描画
458       widthFactor: 0.85,
459       heightFactor: 0.85,
460
461       child: AnimatedRotation(
462         // 石の状態に応じて回転
463         turns: turns,
464
465         // 回転アニメーション時間
466         duration: const Duration(milliseconds: 300),
467
468         // 滑らかな回転を実現
469         curve: Curves.easeInOut,
470
471         child: Stack(
472           children: [
473             // 白石の描画
474             // 裏面として利用
475             Transform.flip(
476               // 上下反転
477               flipY: true,
478
479               child: _buildStoneContainer(
480                 colors: [Colors.white, const Color(0xffdddddd)],
481                 isBlack: false,
482               ),
483             ),
484
485             // 黒石の描画
486             // stateに応じて表示・非表示を切替
487             Opacity(
488               // 黒石なら表示, 白石なら非表示
489               opacity: state == 1 ? 1.0 : 0.0,
490
491               child: _buildStoneContainer(
492                 colors: [const Color(0xff555555), Colors.black],
493                 isBlack: true,
494               ),
495             ),
496           ],
497         ),
498       ),
499     );

```

```

500     }
501
502     // 石本体の見た目を生成する関数
503     Widget _buildStoneContainer({
504         required List<Color> colors,
505         required bool isBlack,
506     }) {
507         return Container(
508             decoration: BoxDecoration(
509                 // 円形の石を生成
510                 shape: BoxShape.circle,
511
512                 // 立体感を出すための影
513                 boxShadow: const [
514                     BoxShadow(color: Colors.black38, blurRadius: 4, offset:
515                     Offset(2, 2)),
516                 ],
517
518                 // グラデーションによる光沢表現
519                 gradient: LinearGradient(
520                     begin: Alignment.topLeft,
521                     end: Alignment.bottomRight,
522                     colors: colors,
523                 ),
524             );
525     }
526 }

```